

SAMG_p
PARALLEL

SAMG

0110011100011100100111000111001001110001

Algebraic
Multigrid Methods
for Systems

Autonomous

Solver Control

**User Manual and
Control File Description**

Document version 2021
June 2021

Service & Technical Support
samg@scai.fraunhofer.de

Fraunhofer Institute SCAI
Schloss Birlinghoven 1
53757 Sankt Augustin
Germany

 **Fraunhofer**
SCAI

1	Overview	3
1.1	Quick Start.....	4
1.2	Acknowledgement	4
2	Usage Mode: Autonomous Parameter Selection.....	5
2.1	Best-Suited Method by Evolutionary Learning	5
2.2	Limit the Initial Search Space by Surrogate Learning	9
2.3	Convergence Monitoring During a Training Phase	10
2.4	Reproducibility	11
2.5	Logging and Output.....	12
3	Usage Mode: Convergence Monitoring and Fallback Strategies	13
4	Usage Mode: Control of the Setup Reusage	14
5	Definition of Methods and Parameters via XML File	16
5.1	XML Syntax Conventions for ASC Control	16
5.2	XML Elements for ASC Control.....	17
5.2.1	Method / Fallback Method / Default Method	17
5.2.2	Parameter.....	19
5.2.3	Value	20
5.3	Example XML File	21
6	Index	22

1 Overview

SAMG offers a huge variety of user parameters that allow for adjusting solution strategies at macro and at a (high) detail level. However, the optimal strategy for a set of problems may not always be obvious to find. And, moreover, even within the same type of application, it may be case-dependent which parameter settings are optimal.

The Autonomous Solver Control (ASC) transfers the final decision on parameter settings from the user to SAMG. The user just has to define a search space of parameters. This is meant to exclude mathematically wrong settings for a type of application and to enforce settings that may be known to be mandatory. Within this search space, however, the Autonomous Solver Control finds an optimal set of parameters for a given test case.

This is realized by methods of evolutionary machine learning with a tree-based evolutionary learning method that is accompanied with a surrogate learning mechanism.

Apart from finding a set of parameters that optimizes the computational performance, this also ensures the robust applicability of SAMG. The Autonomous Solver Control only accepts those parameter settings that provide a robust solution for the given problem(s). It re-considers its decision if robustness problems should occur in the scope of a simulation run.

To maximize robustness at minimal performance impacts, the Autonomous Solver Control as a further use mode allows for a convergence monitoring: If the convergence is not robust enough with a given set of parameters, fallback strategies are applied. This initial set of parameters can either be user-defined, i.e., without the previously described learning strategies, or self-decided based on the provided search space.

This monitoring mechanism with fallback strategies allows for using parameter settings that provide a good performance for the vast majority of cases in a simulator but are known to have a risk to fail in a small amount of cases. For the latter ones, the fallback strategies still provide robust solutions, while the overhead costs of the more robust solver strategies behind the fallbacks do not need to be accepted where they are not necessary to ensure robustness.

Finally, a third usage mode allows for letting SAMG decide whether setups shall be re-used or not. That is, setups will only be computed if they are either user enforced, for instance, due to a new time step, or if SAMG concludes them to be numerically beneficial enough to outweigh the costs to construct them. Again, a monitoring mechanism ensures a setup to be enforced where required to ensure numerical robustness.

All three usage modes, parameter decisions based on machine learning, convergence monitoring and the control of setup reuse, can be activated independently of each other. That is, only one or two of the modes can be used in a particular application with the other(s) remaining deactivated.

If activated together, the different usage modes will interact with each other to optimize towards both or all objectives.

Please note that by default the parameter-selection and the setup-reusage mode consider measured run-times as computational performance. That is, the results are not absolutely reproducible as measured runtimes are always subject to some fluctuation. This does not impact the fallback mode, as this exclusively considers numerical robustness. This (up to round-off) is reproducible.

However, you may turn on a reproducible mode where artificial, pre-described runtimes are considered instead of measured ones. The quality of the results then is directly linked to the accuracy of the pre-described times. Results are absolutely reproducible then, though.

In addition, a semi-reproducible mode is provided that aims at combining pre-described and measured runtimes. This drastically reduces the impact of the fluctuation in measured results, however, it does not fully rely on pre-described values.

1.1 Quick Start

The three usage modes can be simply activated with the following settings, if all other settings shall be at reasonable defaults:

- Usage Mode Autonomous Parameter Selection
First, a set of methods and parameters, i.e., the search space, needs to be provided via XML file. See Chapter 5 for details.
Then, the usage mode is activated by setting *asc_select_method=1*.
In addition, the following parameters should be considered:
 - *asc_no_generation* defines the number of generations in the evolutionary learning process. With a large search space, an increase of the default value may be considered. SAMG will provide a warning message if the value is significantly lower than the estimated number of parameter combinations.
 - *asc_training_phase* should be set to true if no training-on-the-job but a dedicated training phase was requested. Otherwise, the default can remain.
 - *asc_learn* needs to be set to 1 if surrogate learning shall be activated in addition to the evolutionary process.

Details on all further controls regarding the autonomous parameter selection can be found in Chapter 2 below.

- Usage Mode Convergence Monitoring and Fallback Strategies
First, a set of fallback strategies needs to be provided via XML file. See Chapter 5 for details.
Then, *asc_backup* needs to be set to, for instance, 0.95 (if the fallback shall be activated if the average convergence rates exceeds 0.95).
Details on all further controls regarding the autonomous parameter selection can be found in Chapter 3 below.
- Usage Mode: Control of the Setup Reusage
Set *asc_iswit* to 1, 2, or 3, depending on whether you intend to have the setup reuse with *iswit* 1, 2 or 3 (cf. the description of *iswit* in the SAMG User's Manual).
Details on all further controls regarding the autonomous parameter selection can be found in Chapter 4 below.

1.2 Acknowledgement

The development of SAMG's autonomous solver control has been funded by the Fraunhofer Research Center for Machine Learning within the Fraunhofer Cluster of Excellence Cognitive Internet Technologies in 2020 and 2021.

2 Usage Mode: Autonomous Parameter Selection

In the general case, you will need to define a parameter space to be considered by the learning method (cf. Chapter 5). Then you can activate the process by *asc_select_method*:

Parameter	Value	Meaning	Default
asc_select_method	Integer. General control of the parameter selection mode.		0
	0	No parameter selection is performed.	
	1	Perform a parameter selection based on the given parameter space (unless a special mode has been activated; see below). The learning method will be controlled by the parameters that are described in the following sub-sections.	

For some applications, pre-defined parameter search spaces are available in SAMG. These can be activated via special *asc_...* user parameters. An XML file will be ignored then. The SAMG team will provide further information if this applies to your case.

Please note that, by default, decisions are based on measured runtimes. Thus, reproducibility of results cannot be guaranteed, at least not within the pre-defined convergence thresholds. Reproducible and semi-reproducible modes are available, though (→ 2.4).

All decisions will be taken on the host process in the MPI case (rank 0). Decisions will be distributed to the workers.

This is especially relevant regarding files (surrogate learning database → 2.2, log- and output-file → 2.5). If these features are activated, file-IO will exclusively be performed by the host process. By default, all features that require file-IO are deactivated, though.

2.1 Best-Suited Method by Evolutionary Learning

From the provided set of methods, the Autonomous Solver Control identifies the best-suited one in terms of performance via an evolutionary learning approach. In order to improve the runtime, a tree-based structure is applied to the parameter spaces that guides the learning process. This primarily distinguishes different classes of methods (e.g., different AMG approaches, different incomplete factorization methods, etc.), methods (i.e., realizations of such classes) and the different parameter settings within a class. Further separations are handled internally.

Parameters can be either fixed, if a certain setting is mandatory for a certain method, or varied. Variations can be either based on a pre-defined list of options. Or a continuous interval is considered in which the parameter value is selected.

Details of the selection of parameters, methods and classes will be provided in Chapter 5.

During the evolutionary process, certain paths of the tree may be finally discarded if they are found to be inappropriate. Thus, the maximal number of generations may not always be used by the methods. And, analogously, (by far) not all possible parameter combinations are tested. This drastically reduces the computational costs of the parameter selection.

The following main parameters allow for general adjustments and should be considered when using the parameter selection mode.

Parameter	Value	Meaning	Default
asc_training_phase	Logical. Decide whether there will be a training phase for the method selection of ASC.		False
	False	Training will be “on the job”. This <u>requires different linear systems to be close enough</u> , but minimizes overhead costs. <u>Note:</u> If a considered method fails to converge, the same problem is re-run with another method (as long as the max. number of generations is not exceeded).	
	True	Separated training phase for the first linear system. This will be tested with different methods as long as the max. number of generations is not exceeded. That is, some <u>computational overhead</u> is invested in the beginning of an application process. Optionally, the convergence monitoring (→2.3) can be activated when a training phase is used.	
asc_no_generation	Integer. Define the number of generations during the evolutionary process. A higher number of generations will result in a more optimal result of the learning process, at the expense of (potentially) higher computational costs. The provided number is directly related to the size of the parameter search space. <u>Note:</u> SAMG will print out a warning message of the provided number drastically differs from the number that SAMG expects in order to consider all provided parameters. However, runtime restrictions may make a smaller number being a reasonable choice, while also a higher number may be user-requested in order to improve the quality of the learning process.		20
asc_assumed_no_setup_reuse	Integer. In the judgment of the performance results, the respective number of setup-reuses is considered. That is, the cycle-time will be considered stronger in the decisions by a respective factor.		0
asc_recheck	Integer. Decide whether and how to recheck the choice of the parameter selection during the following SAMG calls in an application.		-1
	-1	Recheck only if the current method fails to converge.	
	0	Never recheck the selected method and parameters.	
	> 0	Recheck the parameter selection in the given, fixed interval <i>or</i> if the current method fails to converge.	

In addition to these rather general parameters, the following parameters allow for a more fine-grained control of the evolutionary learning process. This especially covers the way how paths in the learning tree may be emphasized – or discarded. But also the amount of stored data and the handling of interval-varied parameters can be controlled in details. A typical user will not need to change the provided defaults, though.

Parameter	Value	Meaning	Default		
asc_fittest_survival	Integer. Detailed control of the evolutionary learning process and how the fitness of methods shall be evaluated.		1		
	[1:1]	General control			
		0		Switch off evolutionary learning. The process becomes a trial-and-error then.	
		1		Evolutionary learning with reasonable defaults for digits 2 and beyond.	
		2		Evolutionary learning with settings as defined by the further digits.	
	[2:2]	Defines the quantile where a class of methods needs to be in in terms of the performance comparison of methods. That is, with x the digit, a class of methods needs to be among the $1/(1+x)$ amount of methods, if compared by their performance. Otherwise, if this decision already is allowed (see [3:3] below), a class of methods will be finally discarded from the further evolutionary process. Thus, the higher the value, the more clearly a class of methods needs to produce bad results in order to be discarded.			
	[3:3]	In order to be allowed to make a final decision about a class of methods (see [2:2] above), a certain number of runs with methods of that class is required. With x the digit, the amount of $1/(1+x)$ of the number of calls that would be expected (uniform distribution, based on the total number of test runs) is required before a decision is made. Thus, the higher the value, the more clearly a class of methods needs to produce bad results in order to be discarded.			
[4:4]	The analogue to [2:2] for the methods within a class (next level of the learning tree).				
[5:5]	The analogue to [3:3] for the methods within a class (next level of the learning tree).				
asc_vary_chunks	Integer. Controls the discretization of those parameters that shall be tested within a continuous interval rather than a list of options.		10		
	0	No discretization of intervals is performed.			
	>0	Intervals are divided into the respective number of chunks. This will guide the process of finding further settings by focus on searching in chunks that already gave a good performance. Whereas the likelihood of less promising chunks is successively reduced.			
asc_limit_attempts	Logical. If true, the number of attempts per parameter setting is limited such that accidental gatherings of some settings are avoided. This requires SAMG to estimate any possible parameter setting in advance. While this produces a small computational overhead in advance, it improves the quality of results by avoiding redundancies.		True		

	<u>Note:</u> If set to false, no warning can be generated if <i>max_no_generations</i> drastically mismatches the expected maximal number of required attempts.		
asc_limit_attempts_chunk	Logical. If set to true, the analogue limitation applies to those parameters that are varied within a continuous interval. <u>Note:</u> This only applies if <i>asc_limit_attempts</i> is true <u>and</u> <i>asc_vary_chunks</i> is greater than zero.		True
asc_no_archive	Integer. Defines how many results shall be maximally stored per method during the training run. Lower numbers result in less memory consumption. At the expense of lower quality in the results.		10
	>0	This is just a lower bound. It may be extended in accordance with <i>asc_no_generation</i> .	
	0	No information are stored at all. This drastically reduces the quality of the evolutionary learning.	
	<0	This is a fixed number. Less than <i>asc_no_generation</i> test runs may effectively be performed, if the amount of data would exceed the limit otherwise.	
asc_do_not_stop_test_runs	Logical. If true, a training run will <u>not</u> be stopped in the training phase. Even if the best already achieved runtime should have been exceeded. Also convergence monitoring by <i>asc_monitor_convergence</i> (→ 2.3) is suppressed then. This especially has advantages if importance metrics for the varied parameters shall be studied (cf. <i>asc_output</i> with a negative value. → 2.5). Note that this parameters does not have impacts in the internal decisions but only has effects on the accuracy of importance metrics that the user may request for analysis. Also note that the setting <i>true</i> will result in <u>performance overheads</u> !		False
asc_conv_by_eps	Logical. Control how the Autonomous Solver Control shall evaluate a run to have converged or not. This is especially relevant if non-standard convergence criteria are used (cf. SAMG's User Manual).		True
	True	Evaluate successful convergence based on the criterion <i>eps</i> from SAMG's interface.	
	False	Assume convergence iff an SAMG run returns with no error or warning code <u>and</u> the number of performed iterations did not exceed the limit for the number of iterations. Some (<i>eps</i> or a more sophisticated) convergence criterion is fulfilled then.	

2.2 Limit the Initial Search Space by Surrogate Learning

SAMG's Autonomous Solver Control can further limit the space of parameters that shall be considered for the automated parameter selection. This is based on results with previous calls to SAMG and applies a surrogate learning model. Thus, also runs with different simulations can be compared with each other here and SAMG neglects results that are not representative for the current application. This surrogate model compresses information about the linear system as well as about the computation settings, especially parallelism.

By default, because it requires file-IO, the surrogate learning is deactivated. You can easily activate it by setting `asc_learn` to 1. Further, fine-grained controls are available. These can safely remain at their default value for the typical user.

Parameter	Value	Meaning	Default
asc_learn	Integer. Control of the surrogate learning method that is used for the initial state of the evolutionary learning above.		0
	[1:1]	General control digit.	
		0 No surrogate learning is performed	
		1 Surrogate learning is activated with reasonable defaults for digits 2 and beyond.	
		2 Surrogate learning with individual settings for the further options (digits 2 and beyond). Contribute to database and learn from it.	
		3 Surrogate learning with individual settings for the further options (digits 2 and beyond). Contribute to the database but do not learn from it.	
		4 Surrogate learning with individual settings for the further options (digits 2 and beyond). Do not contribute to the database but learn from it.	
	[2:2]	With x the digit, $1/(10-x)$ gives the quantile that will be relevant to learn from. Only the methods/classes from those learned surrogates that are close enough to the current one in terms of this quantile will fully participate in the evolutionary process. For the others, the dropping-likelihood is drastically increased. The higher, the less likely will methods and classes become more likely to be dropped later on.	
	[3:3]	Defines the "closeness" that the current surrogate needs to have to the closest stored one to be considered "identical". "Identical" then implies taking the resp. method and parameters w/out further evolutionary process.	
		0 A surrogate is never assumed to be identical.	
		>0 The bigger, the looser the respective tolerance, and the more likely to skip the evolutionary process.	
asc_learn_database	Character. File for the database of the surrogate learning method. Compatibility with earlier versions of the surrogate model is ensured by the data format.		"samg_db.asc"

asc_learn_database_len	Integer. The absolute value describes the maximal number of entries accepted in the database. This controls the amount of disk space that is occupied by the database.		1000
	>0	If the number is exceeded, first try to drop entries where no known method could be affiliated with. Then proceed with the oldest entries.	
	<0	If the number is exceeded, drop the oldest entries.	

2.3 Convergence Monitoring During a Training Phase

If the evolutionary learning process takes place in a separate training phase (c.f. *asc_training_phase* → 2.1), SAMG's Autonomous Solver Control will discard a method if the convergence is worse than the best already known one. However, if it is known that at least one of the methods of consideration convergences rather robustly for the given type of linear system, the Autonomous Solver Control can also be advised to stop training runs as soon as a certain convergence threshold is hit. This avoids unnecessary training efforts then.

Note:

This is not related to the fallback mechanism (→ Chapter 3). While working comparably, this convergence monitoring only aims at avoiding avoidable work during the training phase. Both methods/controls can be used independently of each other.

Parameter	Value	Meaning	Default
asc_monitor_convergence	Double. Control the monitoring of the convergence during a dedicated training phase of the evolutionary learning process.		0.0d0
	+/-	If positive, a convergence failure will result in an extended training phase (i.e., a re-run), if necessary. Otherwise, the number of training attempts (<i>asc_no_generation</i> → 2.1) is a hard threshold.	
	Int-Part	The respective first number of iterations is ignored in the convergence monitoring (e.g., if they are known to be non-representative due to boundary conditions or such).	
	Float-part	The convergence threshold. If the average convergence factor of those iterations that shall be considered exceeds this limit, the training run is stopped.	
asc_monitor_conv_it_cnt	Integer. Decide for how many iterations in a row the convergence threshold needs to be exceeded in order for a training run to be stopped (zero counts as one). With a negative sign this requirement is limited to the solution phase's start. That is, the respective number of iterations (without ignored ones, cf. int-part of <i>asc_monitor_convergence</i>) must have been performed. Then each iteration counts.		0

Note:

If *asc_do_not_stop_test_runs* (→ 2.1) is true, this convergence monitoring will not have any effect.

2.4 Reproducibility

By default, the Autonomous Solver Control considers real, measured runtimes. This is the metric that counts in the end in terms of performance. However, measured runtimes may always be subject to certain variations from hardware effects. Thus, especially where the performance of different parameter settings and methods is close, the results of the parameter selection may not be reproducible.

The solver control allows to enforce reproducibility by using pre-defined performance metrics for the different methods (→ Chapter 5).

These can either be used exclusively, which results in full reproducibility but limits the consideration of differences between parameter settings to mere numerical effects (i.e., the number of iterations and whether the method converges).

Or measured and pre-defined performance metrics can be combined in a dynamic way that generates a semi-reproducible mode: While the likelihood of identical decisions with two runs of the same parameter settings is drastically increased, it can still not be absolutely guaranteed.

In any case, the logging mechanism (→ 2.5) allows for exactly re-running a sequence of SAMG calls with precisely the same decisions by the Autonomous Solver Control. This allows to systematically trace back the results.

Parameter	Value	Meaning	Default
asc_reproducible	Logical. If true, pre-defined performance metrics per method will be used instead of measured runtimes. Only then absolute reproducibility is guaranteed.		False
asc_semi_reproducible_thres	Double. If greater than zero, the semi-reproducible method is activated (if <i>asc_reproducible</i> is false). Then, if the relative difference between measured times exceeds the value of <i>asc_semi_reproducible_thres</i> , the measured times will be used in comparisons. Otherwise, deterministic data is introduced to the comparison - the stronger, the closer the measured times are (the increase of this strength is stronger than linear with decreasing differences). Which reproducible data is used, is controlled by <i>asc_semi_reproducible_use_art</i> below. The use case is, for instance, that if times are known to vary by, say, 5% on a machine to use the threshold 0.05d0. Note: This still does not yield full reproducibility, but it drastically reduces the impact of noise. Full reproducibility is only given with <i>asc_reproducible</i> .		-1.0d0
asc_semi_reproducible_use_art	Logical. If true, the deterministic data that is used with <i>asc_semi_reproducible_thres</i> above are the same pre-defined performance metrics that would be used with <i>asc_reproducible</i> . However, they would not be used exclusively but in combination with the measured times, according to the threshold and the timing difference. If false, the required number of cycles and, if still necessary then, the number of the trainings run will be used as deterministic data.		False

2.5 Logging and Output

By default, the Autonomous Solver Control prints information about its decision process to the screen output. However, the user can request more detailed output to be also printed either also to the screen and/or to a file. This additional output does not only contain internal details but also importance metrics of parameters for analysis purposes.

Parameter	Value	Meaning	Default
asc_output	Integer. Controls the amount of output that the Autonomous Solver Control provides to the user.		0
	+/-	With a negative sign, importance metrics are computed for each parameter and printed in addition to the output details. These can be used for analysis purposes by the user. In order to do so, setting <i>asc_do_not_stop_test_run</i> (→2.1) should be considered. Note that that the computation of the metrics does come at some <u>performance overhead</u> .	
	0	No additional output. Only standard output.	
	1	Information about parameter selections are provided once after the training finished in ".in"-file format.	
	2	Information about the parameter selections are provided with each call after the training finished.	
asc_output_file	Character. File to write .in-output of ASC selection to (cf. <i>asc_output</i> above). Use "screen" to receive screen output.		"screen"
asc_logging	Integer. Allows to log all decisions of the Autonomous Solver Control and their backgrounds. These information can then be used to re-run precisely the same settings and decisions, also on a different machine. This is essentially meant for debugging and analysis purposes. The logfiles essentially follow the same binary data format as the data base from surrogate learning.		0
	0	Logging is turned off	
	1	Decisions of the Autonomous Solver Control are written to a log.	
	-1	The current run of the Autonomous Solver Control is based on the provided log. That is, actually measured results are ignored and precisely the same decisions are made as in the logged case.	
asc_log_database	Character. File for the log.		"samg_log.asc"

3 Usage Mode: Convergence Monitoring and Fallback Strategies

In order to use the convergence monitoring mechanism of the Autonomous Solver Control, you will need to define fallback methods that will then be activated if convergence thresholds are exceeded.

Note:

The order in which fallback strategies are defined gives a priority in which they are applied. That is, the fallback method that is defined first will be the first attempt to stabilize convergence. If the convergence threshold should then be hit again, the second fallback method is attempted, etc.

Parameter	Value	Meaning	Default
asc_fallback	Double. Controls the convergence monitoring and the fallback to respective methods, where found to be necessary.		0.0d0
	+/-	If positive, also a triggered quasi-residual check (cf. "Quasi-Residual Check" in SAMG's User Manual) will result in the next fallback stage to be chosen. Otherwise, only a slow convergence will be considered.	
	Int-Part	Defines an interval in which the original method shall be re-considered after a fallback method has been selected. Zero means "never".	
	Float-part	The convergence threshold. If the average convergence factor of those iterations that shall be considered exceeds this limit, a run is stopped and re-run with the next fallback stage (if available). This fallback stage will be used from then on - till a possible re-check (→ int-part).	
asc_fallback_stops_iswit	Logical. Decides whether an activated backup mechanism should prevent further attempts of the setup reuse mechanism of the Autonomous Solver Control (→ Chapter 4). Thus, additional stability issues from a potentially inadequate setup are fully avoided for the apparently more challenging linear systems.		true

4 Usage Mode: Control of the Setup Reusage

The Autonomous Solver Control can be advised to limit setups (AMG hierarchy, ILU factorizations, etc.) to when this is expected to pay off (or is necessary for a robust convergence).

Even though there is no mathematical evidence that a previous setup of such methods can robustly be applied to a changed matrix, often enough this works perfectly fine and allows to avoid computational efforts for a new setup. This is especially the case where changes in the matrix are rather small.

The Autonomous Solver Control combines the reuse of setups with a convergence monitoring. This way, setups will be enforced if the matrix changes should turn out to be too drastic. Thus, a robust solution is still provided.

Note:

During training runs of the selection of methods (→ Chapter 2), this is deactivated - and will be applied once a method is selected. A potential reuse of setups can be reflected in the decision process of the parameter-and-method selection by setting *asc_assumed_no_setup_reuse* accordingly.

In the case of a fallback method becoming activated (→ Chapter 3), reuse of setups is stopped by default to avoid interferences (→ *asc_fallback_stops_iswit*).

Parameter	Value	Meaning	Default
asc_iswit	Integer. Controls the reusage method for setups. <u>Note:</u> If activated, the user-supplied setting of <i>iswit</i> will be overwritten by the mechanism.		0
	0	No reuse of setups is induced by the Autonomous Solver Control. The user may still request setup reusages individually by setting <i>iswit</i> (→ SAMG User Manual), though.	
	>0	Reuse a setup as long as it is expected to pay off (extrapolated from the setups costs and the achieved convergence rate with a fresh setup) If convergence fails in the case of reused setup, reusage will be switched off till the next recheck of the Autonomous Solver Control (→ <i>asc_recheck</i> , Section 2.1). Reusing the setup is done with <i>asc_iswit</i> becoming the new <i>iswit</i> (1,2,3,...)	
	<0	As above, but a convergence failure only results in a new setup, not in switching off the reusage for upcoming calls	
asc_iswit_reproducible	Integer. Controls whether real timings or artificial (thus, reproducible) timings shall be used for the setup re-usage control. Semi-reproducibility (→ <i>asc_semi_reproducible_*</i> , Section 2.4) is not possible here because multiple runs may be compared with each other. If selected without auto-controlled methods (→ Chapter 2), the artificial times will be provided in the XML input file (→ Chapter 5) via <i>default_method</i> .		2
	0	Do not enforce reproducibility	
	1	Enforce reproducibility	
	2	Follow the setting from <i>asc_reproducible</i> (→ 2.4)	

asc_iswit _setup_now	Logical. Triggers the execution of a fresh setup in the following call to SAMG independently of whether the solver control would attempt to do so. This can, for instance, be used when the necessity of a new setup is known due to "external circumstances" such as new time steps, grid changes, etc. Whenever this parameter is true, a new setup is performed - and the parameter is reset to false.		false
asc_iswit _convergence	Double. Defines a convergence threshold for those runs where the setup is reused by the Autonomous Solver Control mechanism. If the convergence rate in such a run exceeds the threshold, the run is stopped and a fresh setup will be computed. It will then be continued with the fresh setup (but based on the already computed solution).		0.95d0
	>0	If a fresh setup was enforced, the reusage proceeds in subsequent calls to SAMG.	
	<0	Once a fresh setup was enforced, no ore setup reusages will be attempted by the mechanism.	

5 Definition of Methods and Parameters via XML File

The Autonomous Solver Control needs to know which methods (→ Chapter 2) and fallback methods (→ Chapter 3) it shall consider. Also the according parameters and their (possible) values need to be defined for the control mechanisms.

In the general case, this is realized with an XML-based input file that the user provides. It needs to be provided to the host process (i.e., rank 0). In an MPI run, this will distribute the information to all other processes.

Parameter	Value	Meaning	Default
asc_xml	Character.	Input file (relative to the execution directory).	"samg_asc.xml"

For certain special application areas, methods are pre-defined internally. These settings can be accessed via special asc-parameters. Individual settings cannot be made then, though. The SAMG team would have provided advise if your application fell into this category.

Elsewise, the XML control file will grant flexible access to (almost) any SAMG parameter that can become subject of the ASC control mechanism. This follows the standard XML syntax with some adaptations (→ 5.1). It allows for defining different types of methods (→ 5.2.1). Either simple methods (for parameter selection → 2), fallback methods (for fallback strategies → 3) and/or a default method (for reproducible setup reuse control → 4).

All types of methods are characterized by parameters (→ 5.2.2) that can have (different) values and variation ranges (→ 5.2.3).

Regarding the interplay of Autonomous Solver Control selections and user settings of SAMG parameters, please note:

- If a method, a fallback method or a default method becomes selected by an Autonomous Solver Control mechanism, the respective parameters will overwrite all user-settings that may have been made. Including those via an *samg.in* file.
- The only exception is *ncyc_max* (part of *ncyc*). If *ncyc* should be subject of an Autonomous Solver Control method, all other parts of *ncyc* are subject to the control. However, the maximum number of iterations is always transferred from the current user setting.
- Upon return from an Autonomous Solver Control method, fallback method or default method, all respective parameter settings will be undone. Either to the user setting or the SAMG default, depending on whether the user has accessed parameters or not.
- All SAMG parameters that are not subject of the currently selected Autonomous Solver Control method remain at either their default or, if applied, the user setting.
- If the parameter *smoother* is used with a method, all settings of the smoother control will be stashed – and rewritten upon return. It is impossible to only access parts of the smoother control (some levels, directions, etc.) via the Autonomous Solver Control.
- In MPI applications, the XML control file is read by the main process (rank 0). The information are then distributed internally, as necessary.

5.1 XML Syntax Conventions for ASC Control

The control file for the Autonomous Solver Control essentially follows the XML syntax. That is, different elements (methods, parameters, values → 5.2) can be defined by so-called tags. Tags are encapsulated by angle brackets (< and >). An element starts with the respective tag and has to end with a corresponding end tag (marked by a slash /).

Each element can contain certain other elements (methods contain parameters, parameters contain values). Attributes of an element are defined within the tag definition by an assignment operator =.

Essentially, the scheme reads as:

```
<tag attribute_a="some value" attribute_b="some value">
    <contained_tag attribute="some value">
        ....
    </contained_tag>
</tag>
```

The following conventions holds for the ASC control file:

- Tags are encapsulated by angle brackets: `<tag>`
- Tags are (and have to be) ended by slashes: `</tag>`
It will be described below if some tag may not necessarily need to be ended explicitly
- Multiple tags are allowed per line of the XML document
The line length has to be below 300 chars, though
- There must not be a line break between `<` and the corresponding `>`.
An end tag may be in a different line, though.
- Attributes follow the tag name in arbitrary order.
- If attributes are defined more than once per tag, the latest setting will be selected.
- Attributes are optional to be set - defaults would be used otherwise.
- Between different attributes (or tag name and first attribute), at least one space must be used. More are also allowed.
- Values are assigned to attributes by the operator `"="`.
Quotation marks (`"` or `'`) can be used, but are not mandatory. If they are not used, no spaces are allowed in the attribute's value. If they are used, the starting and ending ones must be identical (i.e., either `"` or `'`).
- Neither tags nor attributes are case-sensitive
For values, only name values are case-sensitive
- If `#` is the first non-space character of a line, this line will be ignored (i.e., a comment).
- A line may have an arbitrary number of heading or trailing spaces.

5.2 XML Elements for ASC Control

Different elements are available for controlling the Autonomous Solver Control. The corresponding XML are described in the following, along with potentially encapsulated elements and used attributes.

5.2.1 Method / Fallback Method / Default Method

The Autonomous Solver Control deals with different types of methods:

- A plain method is used in the autonomous parameter selection (cf. Chapter 2).
The corresponding tag is `<method>`.
The order that is used to define methods is arbitrary and does not have an effect on the parameter selection.
- A fallback method is used if the respective fallback strategy (cf. Chapter 3).
The corresponding tag is `<fallback_method>`.
The order that is used defines the hierarchy of fallback methods: The first fallback method is attempted first, the second only after also the first fallback method fails to converge, etc.
Thus, there will be as many possible fallback stages as fallback methods are defined.
- A default method is used to define artificial timings for the (optional) reproducibility of the setup reuse control (cf. Chapter 4).

The respective tag is `<default_method`.

Only one default method can exist for the Autonomous Solver Control. If multiple ones are defined, the last one is considered.

The plain and fallback method can contain an arbitrary number of parameters. These will be set if the respective method is selected. As outlined above, they will overwrite user settings then and are reset upon return.

For the default method, that only is meant to define artificial timings, parameter settings do not make sense. That is, defining encompassed parameter tags is possible, but they will simply be ignored.

Any kind of method has the following attributes. The respective defaults will be used if they should not be specified.

Attribute	Meaning	Default
name	String. Arbitrary name that is sued for printouts.	"Unknown"
class / descriptor	Class descriptor of method. Used in the tree-based learning of the parameter selection (→ 2.1). Must be one of Undefined, Direct, Relaxation, ILU, AMG. Class or descriptor can be used equivalently with the same effect.	Undefined
signature	Integer. Unique identifying number for the method in the learning and/or logging databases (→ 2.2 and 2.5). Recommendation: The date when the method is defined the first time, in the form YYMMDDHHMM	0
settime settime_n settime_n2	Float. Time that shall be assumed for the setup in the reproducible mode (→ 2.4 and 4). The <i>_n</i> and <i>_n2</i> parameters optionally allow for the influence of <i>nnu</i> . In the reproducible mode, the assumed setup runtime will be $settime + settime_n * nnu + settime_n2 * nnu^2$.	0.0
cyctime cyctime_n cyctime_n2	Float. Analogue settings as settime above, just for the solution phase in reproducible mode. Note that this gives the assumed time per cycle. The actually used number of cycles still is considered!	0.0
range_omp_min range_omp_max	Integer. Optionally defines a minimal and/ or maximal number of threads that need to be active in order for a method to be considered. With a lower or greater number of threads, the method will simply be ignored. Also defining just one limit is possible. The other one is unlimited then. The setting -1 means "unlimited". Note: the order of "range", "omp" and "min/max" is arbitrary. That is: "range_omp_min" is equivalent to "omp_min_range", etc.	-1
range_mpi_min range_mpi_max	Integer. The analogue mechanism for setting a minimal and / or maximal number of MPI processes to be active in order for a method to be considered.	-1
range_nnu_min range_nnu_max	Integer. The analogue mechanism for setting a minimal and / or maximal number of DOF in order for a method to be considered. In the MPI case, this is the global number, cumulated over all processes.	-1

fallback_threshold	Float. Once this fallback stage is activated, the convergence threshold of the fallback mechanism (<i>asc_fallback</i> → Chapter 3) will be replaced by this value. If this value is lower than zero, nothing is replaced. Note: This is only relevant for fallback methods. Otherwise it is simply ignored.	-1.0
---------------------------	--	-------------

5.2.2 Parameter

A `<param` element defines one parameter setting for the encompassing method. Parameters can be either fixed ones: their values will simply be set as defined. Or they can be varying ones: they are varied during the evolutionary learning process (→ 2.1).

Each parameter can contain one or more possible values. Multiple possible values have to be defined for varying parameters to define the set of possible settings.

Before a parameter setting is accessed by the Autonomous Solver Control, the current setting is stored – and rewritten upon return.

Please note that if the parameter *smoother* is accessed by the Autonomous Solver Control, the entire smoother control is stashed – and restored upon return. That is, an ASC method must either not set *smoother* or perform all *smoother* settings (all levels, directions, etc.). It is not possible to only access some *smoother* settings by ASC. The stack-like design of the smoother control does not allow for this. See the SAMG User's Manual for further details on the smoother control.

Please note that first all fixed parameters are set per method, followed by all varying parameters. If parameters are defined multiple times for a method, they are accessed multiple times. The last setting in that order will be the final one then.

Please note that if *ncyc* is defined for a method, *ncyc_max* will still be transferred from the current user input. The ASC setting of *ncyc* only refers to the other parts of that parameter.

Please note that all secondary parameters can be handled by the Autonomous Solver Control, except for those that handle logging. Also ASC parameters themselves cannot be handled by the Autonomous Solver Control (i.e., no recursion is possible). This would simply be ignored. The primary parameters *ncyc*, *nsolve* and *iswtch* are supported.

Please note that currently Character-Parameters are not supported – except for *smoother* and *smoother_dir*.

Unsupported parameters will simply be ignored – and a warning is printed.

The following attributes are possible for parameters:

Attribute	Meaning	Default
name	String. Name of the parameter, as defined in the SAMG User's Manual. If the parameter is known to ASC, the data type is automatically selected. If the parameter is not known (e.g., due to a typo), the parameter will be ignored and a warning is printed.	"Unknown"
mode	Mode descriptor. Can be one of the following three: Fix The first contained value will be selected. List This will be a varying parameter. The variation will be over the list of all contained values.	Fix

	Interval This will be a varying parameter. The variation will be over the interval that is defined by the first two contained values (their min and max, respectively).	
context	String. Type of the derived context for which the parameter setting shall be applied. <i>main</i> corresponds to the main context.	"main"

5.2.3 Value

Each parameter is characterized by one or multiple values. Each value just has the `val` attribute². An end-tag `</value>` is typically not required, but possible. It is mandatory if connected parameters are used, though (see below).

Attribute	Meaning
val	Type depending on the parameter. Value for the parameter. The data type is according to the SAMG parameter (cf. SAMG User's manual). Integer / Double / Float: Provide the respective value. Default is 0(.0). Logical: Provide one of 1/true/t/.true. or 0/false/f/.false.. The default is false. Smoother / Smoother_Dir: Provide the character string as defined in the SAMG User's Manual. The defaults are <i>Clear</i> and <i>Both</i>, respectively.

5.2.3.1 Connected Parameters

The Autonomous Solver Control allows for so-called connected parameters. That is, certain parameter settings are only applied if a parent parameter has a certain value. The parent parameter has to be a varying one, connected parameters would be pointless for fixed parent parameters. Connected parameters can then either be fixed. That is, a certain setting of the parent parameter induces a setting of the connected one. Or they can be varying themselves. That is, some parameter is only varied if the parent parameter has a certain value.

The following example illustrates the setting for the case where *max_agg_size* shall only be varied between 9, 12 and 15 if *ncg* is 179:

```
<param name="ncg" mode="list">
...
<value val="179">
  <param name="max_agg_size" mode="list">
    <value val="9"><value val="12"><value val="15">
  </param>
</value>
...
</param>
```

² Note: you may simplify the setting: `<value val="1">` is equivalent to `<value "1">`

Please note:

- When a connected parameter is used for a value, an end-tag `</value>` for that value is required.
- An arbitrary number of connected parameters is allowed per value.
- A connected parameter is set right after the corresponding parent parameter has been set. First all fixed and then all varying connected parameters of that value of the parent parameter will be processed.
- Connected parameters can overwrite settings that previously have been set (e.g., a fixed parameter).
- A connected parameter cannot have another connected parameter itself.
- Connected parameters can only related to one single value of the parent parameter.
- Each value of a parent parameter can have a list of connected parameters. These can also be the same ones – they need to be explicitly stated per value of the connected parameter, though.

5.3 Example XML File

To illustrate the afore-mentioned, here is an exemplary XML control file that evaluates single-level Jacobi, Gauss-Seidel and ILU – the latter in mixed and double precision mode.

```
<method name="Relaxation" class=relax signature=2101061119 setttime="0.5" cyctime="1.0">
  <param name="levelx" mode=fix>
    <value val="1">
  </param>
  <param name="Smoother" mode=list>
    <value val="Gauss Seidel">
    <value val="Jacobi">
  </param>
</method>
<method name="ILU(0)" class=ilu signature=2101061358 setttime="2.0" cyctime="1.3">
  <param name="levelx" mode=fix>
    <value val="1">
  </param>
  <param name="smoother" mode=fix>
    <value val="ILU">
  </param>
  <param name="ilu0_single_prec" mode=list>
    <value val="0">
    <value val="1">
  </param>
</method>
```

6 Index

<default_method 18
<fallback_method 17
<method 17
<param 19
asc_assumed_no_setup_reuse 6
asc_conv_by_eps 8
asc_do_not_stop_test_runs 8
asc_fallback 13
asc_fallback_stops_iswit 13
asc_fittest_survival 7
asc_iswit 14
asc_iswit_convergence 15
asc_iswit_reproducible 14
asc_iswit_setup_now 15
asc_learn 9
asc_learn_database 9
asc_learn_database_len 10
asc_limit_attempts 7
asc_limit_attempts_chunk 8
asc_log_database 12
asc_logging 12
asc_monitor_conv_it_cnt 10
asc_monitor_convergence 10
asc_no_archive 8
asc_no_generation 6
asc_output 12
asc_output_file 12
asc_recheck 6
asc_reproducible 11
asc_select_method 5, 6
asc_semi_reproducible_thres 11
asc_semi_reproducible_use_art 11
asc_vary_chunks 7
asc_xml 16
Method class 18
Method cyctime[_n[2]] 18
Method descriptor 18
Method fallback_threshold 19
Method name 18
Method range_mpi_[min/max] 18
Method range_nnu_[min/max] 18
Method range_omp_[min/max] 18
Method settime[_n[2]] 18
Method signature 18
Parameter context 20
Parameter mode 19
Parameter Mode Fix 19
Parameter Mode Interval 19
Parameter Mode List 19
Parameter name 19
Value val 20